https://hjfy.top/arxiv/2606.19980

ENPIRE：真实世界中智能体机器人的策略自提升

Wenli Xiao[1,2*], Jia Xie[2*], Tonghe Zhang[2*], Haotian Lin[2*], Letian "Max" Fu[3], Haoru Xue[3], Jalen Lu[2]
Yi Yang[2], Cunxi Dai[2], Zi Wang[1], Jimmy Wu[1], Guanzhi Wang[1], S. Shankar Sastry[3], Ken Goldberg[3]
Linxi "Jim" Fan[1,†], Yuke Zhu[1,†], Guanya Shi[2,†]
[1]NVIDIA [2]CMU [3]UC Berkeley [*]Equal contribution [†]Equal advising

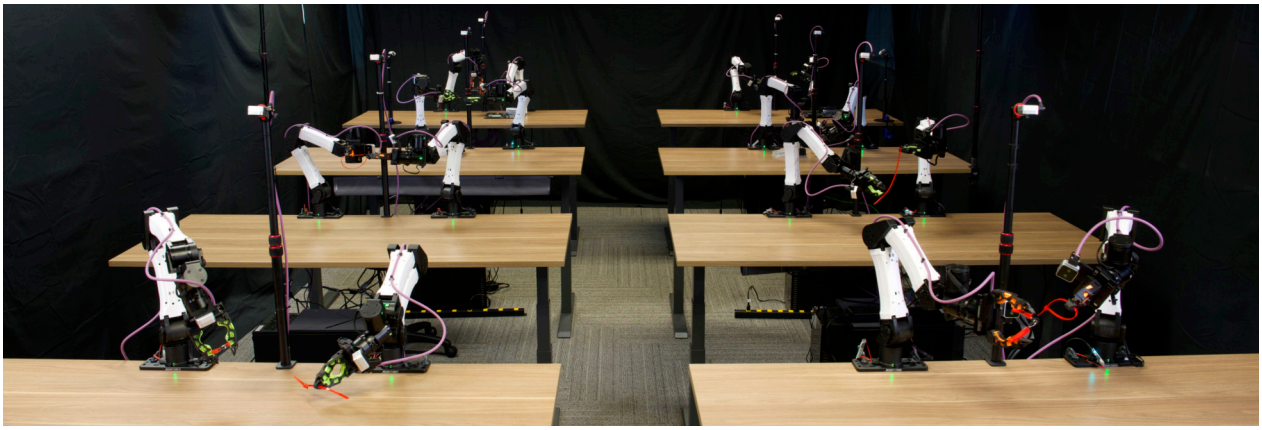


Figure 1: Robot fleet for physical autoresearch. The fleet contains eight bimanual YAM robot stations. Each station owns its robot hardware, compute, and coding agent. Website: research.nvidia.com/labs/gear/enpire

Abstract

在真实世界中实现灵巧的机器人操作，严重依赖人工监督和算法工程，这已成为追求通用物理智能的核心瓶颈。尽管新兴的编码智能体能够生成代码以自动化算法搜索，但其成功仍主要局限于数字环境。我们推测，自动化机器人研究所缺失的关键抽象是针对现实世界策略改进的可重复反馈环：重置场景，执行策略，验证结果，并优化下一次迭代。

为弥合这一差距，我们提出了ENPIRE，一种面向编码智能体的集成框架，通过四个核心模块实例化这一物理反馈流程：一个环境模块（EN），用于自动重置与验证；一个策略改进模块（PI），用于启动策略优化；一个滚动模块（R），支持单个或多个物理机器人并行评估策略；以及一个演化模块（E），其中编码智能体分析日志、查阅文献、改进训练基础设施与算法代码，以应对失败模式。该闭环系统将真实世界的机器人学习转化为可控制的最优化过程，使智能体能够自主管理，从而最大限度减少人工投入，同时支持对不同训练方案与智能体变体的公平消融实验。借助 ENPIRE，前沿编码智能体可自主开发策略，在真实世界中实现挑战性灵巧操作任务高达 99% 的成功率，例如 PushT、将钉子整理至钉盒中，以及使用切割工具剪断扎带。编码智能体可采用多种PI策略进行策略改进，如启发式学习、工具调用、行为克隆、离线或在线强化学习。此外，ENPIRE 可在机器人集群上显著加速，我们提出两个指标——Mean Robot Utilization（MRU）与Mean Token Utilization（MTU），用于衡量多智能体物理自研究的效率。我们在 RoboCasa 中还提供了仿真结果。我们的发现表明，一条切实可行且可扩展的路径正通向真实世界中机器人技术的自主推进。

1. 引言

在迈向通用物理智能的道路上，自动学习灵巧操作技能一直是一个主要障碍。前沿的策略训练方法虽然非常有效，但仍依赖于人类在幕后参与数据采集、评估与重置以及算法调整的生命周期 (19; 20; 24)。随着我们在真实世界中扩大策略学习的规模，人类在策略改进过程中的繁琐劳动不可避免地成为机器人掌握灵巧性速度的关键限制因素。

近年来，由代码智能体驱动的自主研究 (22)（autoresearch）取得了显著进展，展现出自动化算法改进的

© 2026 NVIDIA. All rights reserved.

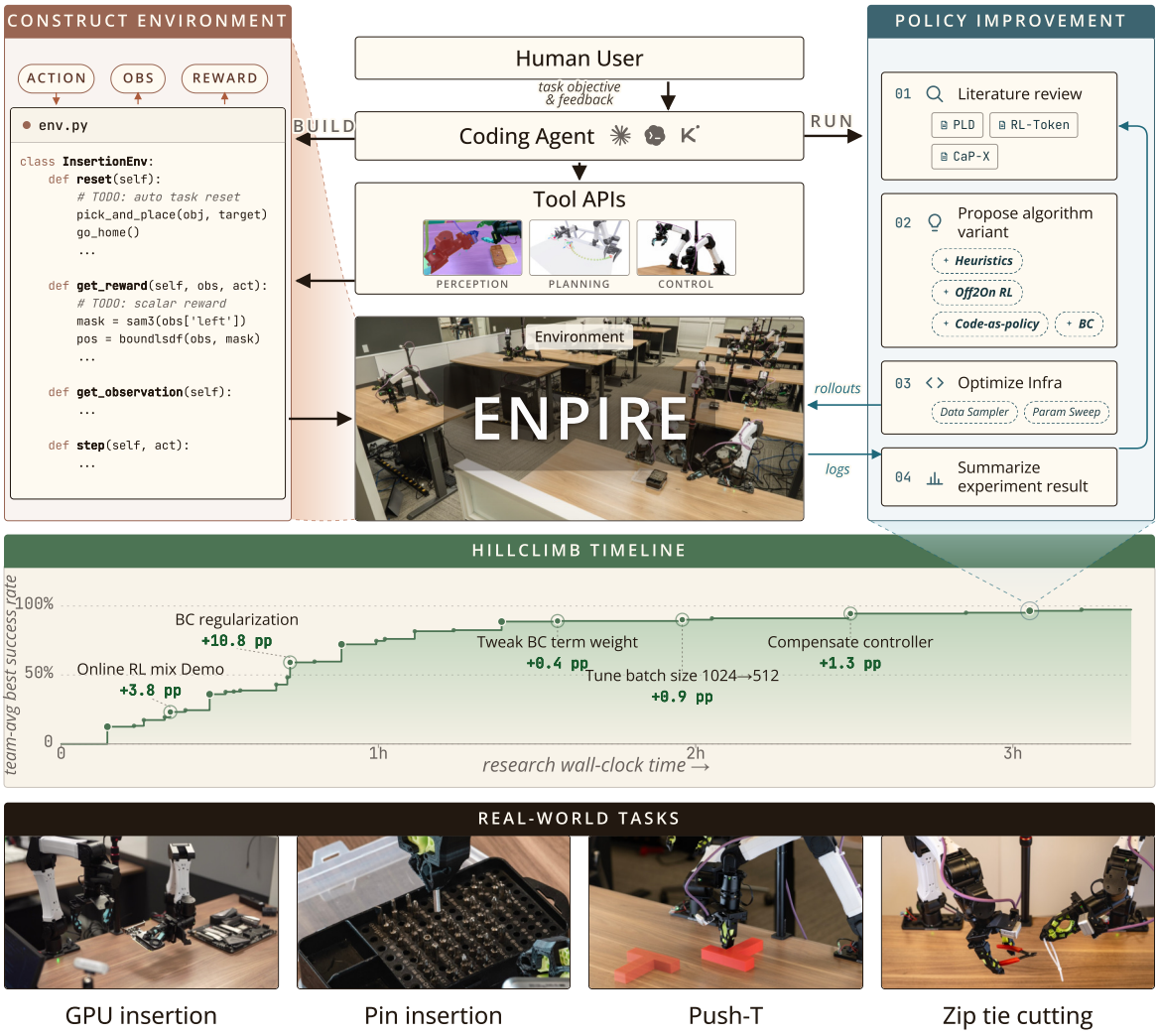


Figure 2: 物理自主研究框架 ENPIRE 的概述。为了解决真实世界中的灵巧任务，ENPIRE 首先利用工具调用根据人类反馈构建一个具备自动重置和验证机制的环境。随后，编码智能体调用环境 API，基于真实世界的奖励信号进行自主研究，并逐步提升策略的成功率。更多结果详见 research.nvidia.com/labs/gear/enpire。

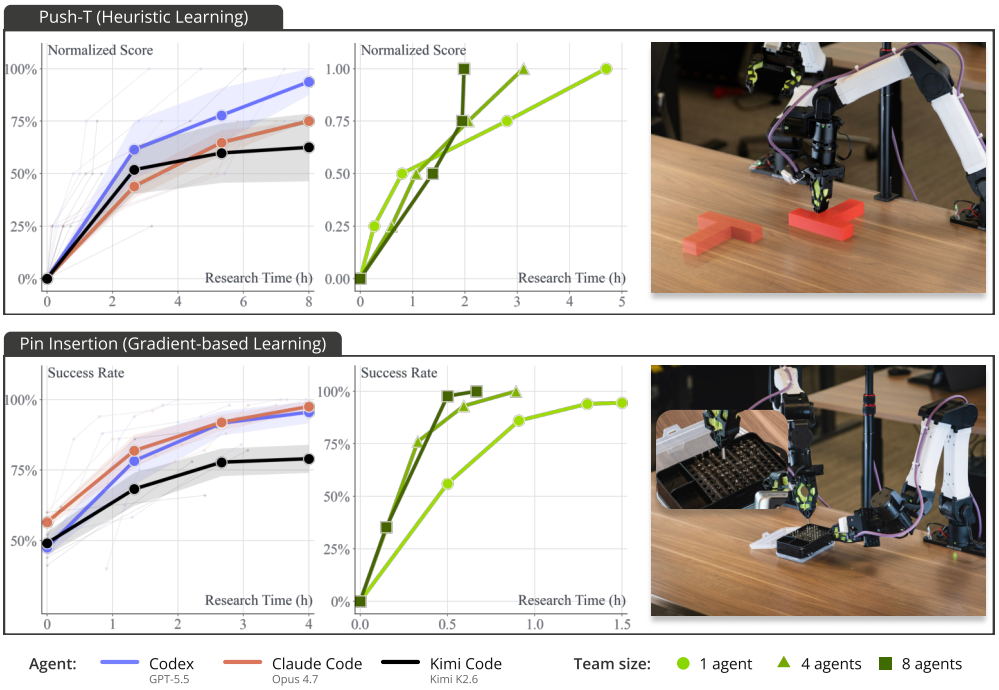


Figure 3: **物理自主研究中编码智能体的基准测试**。ENPIRE 使得 最先进的编码智能体能够在 Push-T 和 Pin Insertion 任务上实现自主策略改进。ENPIRE 还具备资源可扩展性，随着机器人智能体工作进程的增加，达到相同任务性能所需的墙钟时间得以减少。

广阔前景。然而，当我们将这一范式扩展至自动化真实世界策略学习时，却面临独特的挑战。首先，代码智能体缺乏一套用于物理世界闭环假设检验的真实世界环境接口，这需要实现策略的自动化部署、评估以及场景重置。其次，当在机器人舰队上缩放 autoresearch 的吞吐量时，如何在保持资源利用效率的前提下选择并验证假设，仍是一个开放性问题。

为应对这些挑战，我们提出 ENPIRE ，一种智能体赋能框架，通过一系列自主环境接口实现可扩展的物理自研。ENPIRE 将灵巧操作技能的获取分解为两个自研过程。在第一阶段（ENPIRE 中的 EN），编码智能体从人类反馈中构建自主环境接口。通过这一初始环，智能体实现并优化程序化工具调用，建立安全边界、自动重置以及验证流程以完成特定任务。这些流程在离线状态下一次性设置并优化；一旦确定，便作为不可变的 API，在后续阶段重复使用。第二阶段（ENPIRE 中的 PIRE）则过渡到完全自主的自研过程，编码智能体基于真实世界反馈不断优化策略。在环境自动化验证信号的全程引导下，智能体独立探索并优化多种方法，以最大化真实世界任务的成功率，且无需人工干预。

为了将此流水线并行扩展到多个机器人，ENPIRE 引入了一种演化选择假设的机制，以加速策略改进。我们派遣一个去中心化的智能体团队异步测试训练配方，根据平均成功率共享和放弃想法。累积的知识可以迁移到类似的新任务中。

我们将本文的贡献总结如下：

- 我们将灵巧操作的物理自研过程形式化为代码智能体和机器人学领域的一个独立问题。我们提出采用两步法来解决这一问题：首先，通过人工引导的自研过程构建自动环境反馈并进行离线验证；然后，以在线方式执行完全自主的自研过程，利用真实世界反馈来优化策略。
- 我们实现了 ENPIRE ，这是一种智能体框架，通过该框架，编码智能体能够开发可重用的机器人工具，利用过程化工具调用构建奖励机制和重置机制，并在真实机器人上无需人工干预即可有效优化学习算法。
- 我们展示了 ENPIRE 可以在多个灵巧操作任务中逐步提升成功率至较高水平。在插销任务中，策略收敛至 100% 的速度优于前沿的人机协同方法 (48)。我们还观察到在机器人集群上并行自研究的初始缩放行为，并提出了两个关键指标——平均 token 利用率（MTU）和平均机器人利用率（MRU），用于基准测试该过程的效率。

2. 方法

在本节中，我们提出了一种工具设计，使智能体能够执行物理世界中的自动化研究。该过程分为两个阶段：从人类反馈构建环境（ENPIRE 中的 EN）和从真实世界反馈进行自动策略改进（ENPIRE 中的 PIRE）。

2.1. 第一阶段：基于人类反馈的环境构建

为使编码智能体能够自主进行物理自主研究，我们首先需要构建一个对智能体友好的物理交互与反馈抽象，将其作为结构化环境。这包括用于支持长期运行的任务特定安全约束、用于反馈和信用分配的实时自动验证过程，以及用于确保快速迭代的鲁棒自动重置机制。为提高可靠性，智能体通过程序化工具调用构建环境 API，并根据人类评估优化其实施。人力成本为一次性投入，将在后续Sec. 2.2自动策略改进过程中，通过所有机器人的所有实现进行分摊。

硬性安全约束

我们限制机器人的配置空间和运动学行为在安全操作范围内。这些安全区域足以完成任务，并作为硬约束：违反这些限制将立即导致任务失败并自动重置。这为现实世界的交互提供了保障，同时也作为回合终止或截断的来源。

自动验证

在真实世界实验中，机器人学习需要通过传感器输入进行实时验证，以量化实验进展。为了最小化人工工程工作量，编码智能体被指派从过程性工具调用中合成二元奖励函数，以区分任务结果。在仅有几分钟的成功与失败示范的情况下，智能体利用视频和本体感知记录，在最大化预测准确率得分的同时，最小化处理延迟。例如，自动研究发现了一种基于视觉对齐、末端执行器高度和力估计的鲁棒奖励函数，适用于插销插入任务。编码智能体还展示了其在Fig. 4中的扎带插入任务中设计依赖感知的奖励函数的能力，并将其推理延迟优化至 150ms 以下，接近人类视觉系统的反应速度 (43)。更多细节见附录。

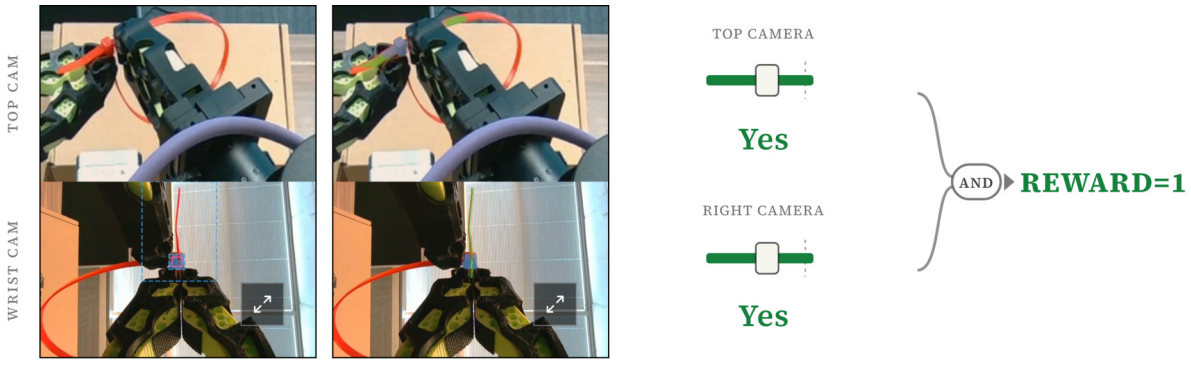


Figure 4: 拉链扣插入奖励。通过裁剪和图像分割测试拉链扣带是否穿过拉链扣头。考虑两个相机视角以防止假阳性。

自动复位

一旦任务完成或失败，智能体将执行一系列工具调用，以将环境恢复到初始状态。对于接触丰富的任务，我们采用受 CaP-X (15) 启发的程序化工具调用，利用模块化操作技能直接将环境重置到最具挑战性阶段的起点。通过将机器人定位在关键动作（如插入插针、安装 GPU 或抓取剪刀）的精确起始点，我们战略性地将学习系统聚焦于这些高精度瓶颈。

安全约束、自动验证和自动重置构成了我们的环境模块（EN in ENPIRE ）。策略还接收视觉-本体感觉传感器输入，并向机器人控制器提交动作指令，这与我们的奖励一起构成了滚动模块（R in ENPIRE ）。构建

完成后，编码智能体可通过不可变的 Gym API (7) 访问这些模块，这些 API 提供反馈信号和调试信息，用于自动化策略改进。

2.2. 第二阶段：从现实世界反馈中实现策略改进

第二阶段利用自动化研究来为特定任务训练策略。初始化时，编码智能体接收任务描述，最终目标是通过自主实验最大化任务的成功率。为了实现这一点，智能体被授予对精简训练代码库的写入权限，该代码库支持基本的端到端策略训练和基于代码的策略合成。

一个激励性的例子

我们通过一个钉子插入任务来说明这一过程，其中策略必须将一个钉子插入一个具有 4 毫米紧密公差的孔中 (Fig. 2)。在这一自动化研究阶段，智能体通过策略改进模块（PI in ENPIRE ）运作，该模块会查阅文献以生成见解、提出假设，并直接修改训练代码——例如行为克隆或强化学习算法——以基于真实世界的自动验证结果优化性能。为了收集丰富的证据，智能体调用环境 API 来记录机器人轨迹、视频录像和奖励信号，同时分析这些统计量以指导持续改进。

通过多智能体缩放加速自动研究

尽管自动重置与验证机制提升了单个部署环的可扩展性，但通过启动并行的多智能体去中心化协作协议，物理自研究过程可进一步加速。该协议在 N 台物理机器人上部署 N 个智能体，以异步方式测试 N 个假设。每个智能体均源自相同的基准策略训练代码库，通过 Git 自主协作，实现此演化模块（E 在 PIRE 中）的缩放。无需人工干预，智能体自发地从同伴处挑选、复制或合并成功的训练配方，以优化其代码搜索。实验表明，显著增加并行智能体-机器人对的数量可大幅减少发现高成功率策略配方所需的实际时间，如图 3 所示。

为了量化编码智能体将其分配的物理资源转化为研究进展的效率，我们提出了两个补充任务级结果的利用率度量指标。平均机器人利用率（MRU）是指在研究的时钟时间中，机器人正在执行实验的时间占比。GPU 利用率是指在研究的时钟时间中，GPU 处于活跃使用状态的时间占比。一个理想的资源饱和智能体将使这两个指标趋近于 1；但在实际中，我们评估的任何前沿编码智能体均未达到该值。为了衡量智能体团队将 token 转化为成功机器人策略的效率，我们定义平均 token 利用率（MTU）为自研究过程中整个舰队的平均 token 消耗量。随后，我们计算了 token 与成功之间的比率，以反映任务学习的 token 效率。图 7 展示了机器人和 token 利用率随智能体团队规模变化的实证结果。

3. 实验

我们展示了 ENPIRE 是一个功能强大且可扩展的 Agentic 自研究框架。

- 首先，ENPIRE 在环境构建后支持自主策略学习。
- 其次，成功率达到收敛的速度 ENPIRE 随着机器人和 token 资源的变化而变化。

我们将重点关注以下灵巧任务，这些任务需要从感知反馈中实现精确且具有响应性的控制：Push-T (12)，机器人通过非抓取运动将 T 形块对齐至目标区域；Pin insertion，机器人通过将插针插入直径为 4 毫米的孔洞来整理插针盒；GPU-insertion，机器人将 GPU 芯片安装到主板上的细槽中；以及 Ziptie-cutting，机器人抓起并闭合剪刀以剪断扎带的尾部。这些任务的可视化展示见图 2。

我们将成功定义为在固定重试次数（此处为八次）下，单次展开中完成任务的概率。与通过独立同分布的 best-of-N 采样衡量成功率不同，我们的重试是在观察到先前失败后进行的，这意味着我们的指标不仅捕捉了准确率，还反映了在环境不确定性面前的上下文恢复能力。我们强调，对于高度灵巧的任务而言，恢复能力比单纯的 one-shot 准确率更为重要，这体现了在真实世界中实现可靠部署所需的鲁棒性。

我们使用 ENPIRE 在多种策略学习范式中开展自动研究。编码智能体可以自由选择各种方法及其组合来解决任务，包括端到端神经网络训练，如行为克隆（BC） (36) 或真实世界强化学习（RL） (18)，以及无需梯度的学习方法，如启发式学习 (47) 和基于代码的策略合成 (25)。我们的实际机器人平台是双臂 6-自由度的 YAM 机器人。我们在物理自动研究实验中对三种编码智能体进行了基准测试，分别是 Codex 搭载 GPT-5.5 xhigh (35)、Claude Code 搭载 Opus 4.7 High (3)，以及 Kimi Code 搭载 Kimi K2.6 思考模式 (33)。

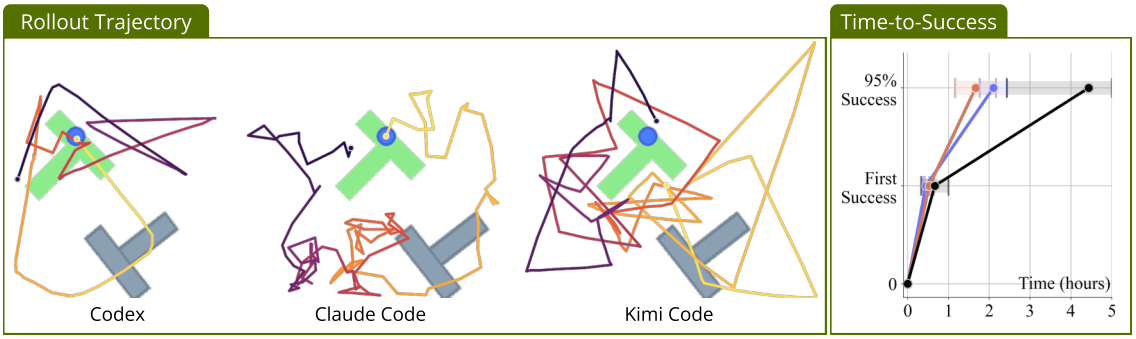


Figure 5: **仿真中的自主启发式学习**。在 Gym-PushT (10) 基准测试中，Claude Code（橙色）和 Codex（蓝色）在约 2 小时内达到 95% 的成功率，而 Kimi Code（黑色）所需时间是其两倍。

3.1. 启发式学习的自研

我们测试了 ENPIRE 在最简单形式下驱动自动策略改进的能力：通过合成感知与控制工具调用 (25) 来学习启发式规则 (47)。为此，我们构建了一个真实世界的 Push-T 环境，并为其配备了仿真版本以进行对比。

物理自研究的独特挑战

如图5和3所示，所有编码智能体在仿真中均成功通过启发式学习解决了 Push-T 任务。然而，在真实环境中，挑战显著增加，导致三个智能体中有两个失败。尽管模拟器提供了稳定、可预测的物理特性，适用于低方差的假设检验，但真实世界条件具有非确定性和时变性：诸如机器人动力学、接触摩擦以及物体运动等因素本质上难以预测，并且在不同试验和硬件之间存在差异。

为了提升现实世界中的鲁棒性，在后续任务中，我们鼓励智能体探索甚至结合多种学习方法，涵盖启发式方法和基于梯度的学习，以应对实际部署中的边缘情况。我们还提出将物理智能体团队进行缩放，以在多样且非确定性的现实物理环境中测试假设。这些设计细节将在后续章节中详述。

3.2. 基于梯度的策略改进自动研究

除了支持启发式学习外，ENPIRE 还能够在精度要求高的任务——插针任务中训练端到端策略，如图 3 所示。在此任务中，编码智能体需要在真实世界评估中连续实现 50 次成功。智能体测试了多种方法来改进策略，包括行为克隆（BC）、结合在线回放数据聚合的迭代 BC，以及带有 BC 正则化项的在线、离线和离线转在线强化学习。智能体还调整了批量大小、演员-评论员策略更新率以及 BC 项超参数等参数。

3.3. 机器人集群的缩放策略学习

我们进一步研究了增加机器人和智能体的数量是否能够减少达到相同目标任务性能所需的壁钟时间。我们为每台机器人分配一个编码智能体，并在该机器人集群上设置相同的环境接口、奖励函数和重置机制。我们考虑在推移任务（Push-T）和插针任务中采用 1 台、4 台和 8 台机器人的集群规模。

在 Push-T 中，智能体数量从 1 个增加到 8 个，达到 1.0 规范化得分所需时间从约五小时减少到两小时。在插针任务中，智能体数量从 1 个增加到 8 个，达到接近完美的成功率所需时间从超过 1.5 小时减少到约 40 分钟。这表明 ENPIRE 能够通过分布式假设选择，将额外的机器人资源用于加速策略改进。

在多智能体设置中，ENPIRE 还可以利用基于代码的策略，在重置期间自动应用领域随机化。例如，GPU 插入过程中的空间配置变化范围显著大于先前工作 (48)，这增强了策略的鲁棒性。

3.4. 通过 Agentic 持续学习转移自动研究经验

我们观察到，多智能体物理自研过程中积累的洞察同样可迁移至类似且新颖的灵巧操作任务。在针插入任务的自主探索阶段结束后，智能体被提示记录并反思其训练配方的演化过程。当我们为 GPU 插入任务启动新

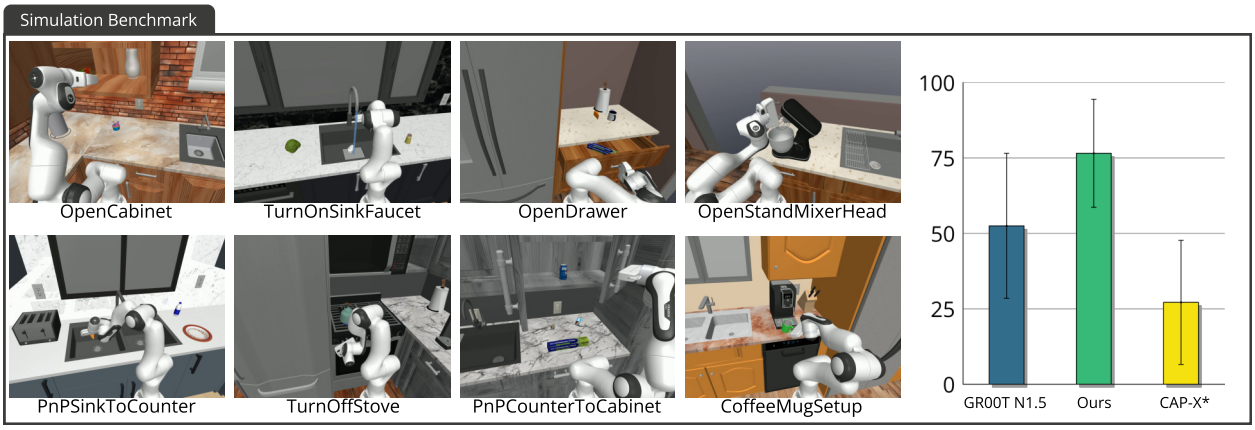


Figure 6: **仿真结果**。在 RoboCasa365 (34) 基准测试中，ENPIRE 在端到端 VLA（GR00T (5)）和无需自研究的 zero-shot Agentic 工具使用（CaP-X (15)）上表现更优。

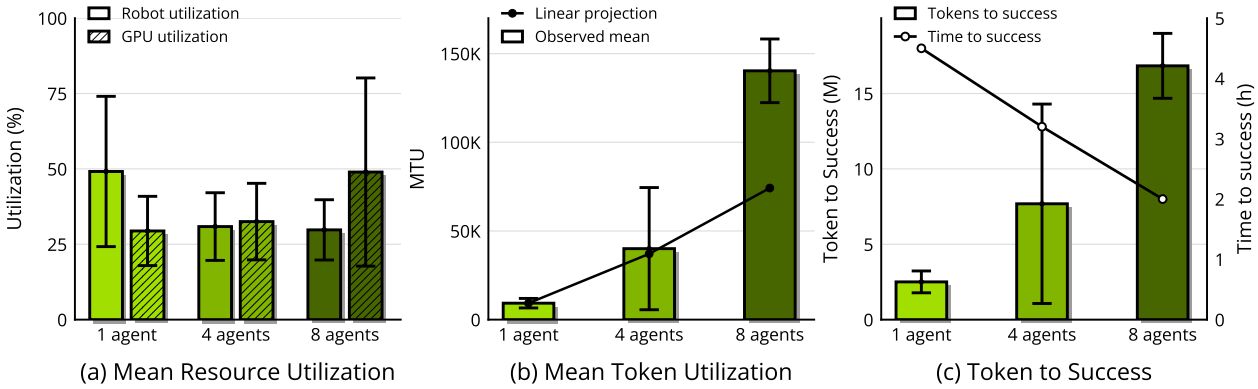


Figure 7: **智能体资源利用的量化**. 从 1 个智能体扩展到 8 个智能体，会提高 GPU 利用率和 token 消耗，但会降低每个机器人的利用率。

一轮自研时，将这些知识附加到新任务的指令中，使得编码智能体能够实现较高的成功率。详细分析见附录。

3.5. ENPIRE 发现基于代码的策略与视觉语言模型之间的协同作用

除了端到端或启发式训练之外，ENPIRE 可以自动整合视觉-语言-动作模型（VLAs）(8) 与过程性工具调用，实现长时程操作。在 RoboCasa365 模拟器 (34) 中，智能体通过使用运动规划和检测工具，在抓取前悬浮于物体上方，从而提升了 GR00T VLA (5) 的成功率。我们已成功将这一策略迁移到真实世界，其中智能体学成在剪刀上方悬浮、抓取并剪断拉链扣，如 Fig. 2 所示。

3.6. 量化智能体资源利用率

Fig. 7 总结了 ENPIRE 在针插入任务中，单智能体、四智能体和八智能体舰队的资源缩放行为。我们使用上述定义报告平均机器人利用率（MRU）和 GPU 利用率。此外，我们测量平均 token 利用率（MTU），定义为每分钟消耗的 token 数量。我们还测量了达成成功的 token 数量和成功所需时间，用于量化完成自研目标所需的 token 预算和实际耗时。

4. 局限性

机器人和计算资源利用率低下

当智能体在读取日志、编写代码、调试或等待语言模型主干时，无法充分利用机器人资源。此外，随着机器人数量的增加，机器人资源利用率（MRU）下降，而 GPU 活跃利用率上升（Fig. 7a）。与单机器人设置相比，智能体活动组成的差异导致智能体团队花费更多时间总结同伴分支，而实际操作机器人的时间减少。智能体还可能无法启动并行训练会话以耗尽 GPU 资源。

token **成本随车队规模呈超线性增长**。

随着舰队规模的增加，token 使用量的增长速度超过了理想的线性趋势。在不超过四个智能体时，MTU 仍接近线性预测值，但在八个智能体时急剧上升（Fig. 7b）。获得成功策略所需的总 token 预算也呈现出相同趋势，其增长速度远超过相应减少的时钟时间（Fig. 7c）。因此，增加舰队规模是以牺牲 token 效率为代价换取更快的策略改进：更大的舰队能更早达到成功，但需要不成比例地更高的 token 预算。

5. 相关工作

5.1. 智能体与代码即策略

这一研究方向建立在一个核心抽象之上——*可执行代码作为智能体的动作*：模型生成程序，运行时返回结构化反馈，环路持续迭代。Code-as-Policies (25) 与 ProgPrompt (41) 将这一范式引入机器人领域，将感知与技能 API 组合为任务计划，而视觉推理则将这种组合思想延伸至视觉模块 (42)。后续研究将单次生成扩展为由执行反馈驱动的多轮环路——包括推理与动作的迹 (51)、自我批判与迭代修复 (32; 40)、推理时搜索 (52) 以及学成工具调用 (37)；Wang et al. (45) 认为代码比语言更适合作为动作表示，因为每一步都可在运行时验证。另一条研究路径则形式化了 *智能体-计算机接口*，即一个沙盒化的外壳环境，智能体可在其中读取、运行并调试代码，如今已成为前沿编程产品背后的核心机制 (3; 33; 35)。

在机器人学中，早期的代码即策略系统暴露了高层、人工设计的 API——闭式技能、语言条件的抓取原语以及可利用性得分 (1)，让智能体负责任务分解而非低级控制的合成。Wang et al. (44) 通过自我验证和自动课程学习构建了一个可复用的 Minecraft 技能库，其中试验成本几乎为零；Fu et al. (15) 发现多轮反馈、技能合成以及集成采样能显著提升相对于低级原语的操作可靠性。另一相关研究方向利用大模型生成辅助训练信号：奖励函数 (31; 49; 53)、仿真到现实的迁移协议 (30)、仿真环境 (46) 以及数据采集流水线 (2)。

5.2. Agentic 自我提升

一个自我改进环路需要每次试验的成本足够低，以支持大规模重复；不同系统之间的主要区别在于每次试验返回给智能体的内容。Ellis et al. (13) 设定了后续系统所基于的保留模式：每个成功求解的合成任务都会向一个不断增长的库中添加命名子程序，这种机制是可行的，因为执行是免费的。Wang et al. (44) 将这一思路引入具身情景，将成功信号替换为大语言模型的自验证以及自动课程——同样可行，因为 Minecraft 的部署成本为零。Yu et al. (53)、Ma et al. (31) 和 Xie et al. (49) 返回奖励而非技能：大语言模型提出稠密奖励，基于该奖励训练策略，并根据训练统计量对奖励进行修正——这一环路通过每分钟数千次 Isaac Gym 仿真实现闭环。Ma et al. (30) 通过合成领域随机化参数迈向硬件，但仍保持在仿真中迭代，并仅在修订停止后才部署；在实验情景层面，Wang et al. (46) 在仿真中生成新任务和资产，而 Ahn et al. (2) 则协调移动机器人舰队进行离线数据采集，而非在环路内迭代。在所有情况下，环路都建立在低成本基底之上，而真实机器人执行仅作为仿真到现实或数据采集的目标，从不作为迭代的媒介。我们保留这些技能积累与奖励生成机制，但直接在硬件上运行环路，此时关键约束资源是智能体的机器人访问预算，而非其计算资源。

5.3. 自主研究智能体与科学发现

最终的研究成果实现了对研究环本身的自动化，最好结合*实验所启动的媒介*来阅读。在大模型出现之前，自主系统已在真实实验室硬件上闭环完成了“假设—实验”环 (9; 23)。而大模型时代下的系统则端到端地自动化了*数字环* (26; 38; 39; 50)；其中一条近期研究脉络正通过实验室自动化 (6; 29) 或由人工执行的湿实验验证

(16)，重新延伸至物理科学领域。评估工作亦体现出这种对数字环节的侧重：MLE-bench (11) 对机器学习工程智能体及其资源缩放能力进行得分；而对 SWE-bench (21) 的分析则警示，性能提升可能源于数据污染而非真实能力提升。由此引出两个空白点：其一，尚无系统能在明确的硬件预算约束下，自主运行并优化物理机器人环——经典机器人科学家固定实验装置且从不自行编写工具，而大模型研究智能体则从未接触真实机器人；其二，尚无基准用于衡量稀缺物理资源的利用效率，而非仅评估能力或单篇论文成本。本系统通过在真实机器人上运行并优化实验环，并为此类预算受限场景引入资源利用率指标，从而填补上述两项空白。

6. 致谢

我们感谢众多同事的帮助，正是由于他们的支持，本项目才得以实现。感谢 Jason Liu、Tony Tao、Tairan He、Alex Lin、Jim Yang、Paul Zhou 和 Abhi Maddukuri 提供的深刻讨论与反馈；感谢 Yide Shentu、Bike Zhang、Angchen Xie、Dvij Kalaria 和 Yuqi Xie 在实验方面的支持；感谢 Lion Park、Matin Furutan、Jeremy Chimienti、Dennis Da 和 Tri Cao 在车队运营方面的贡献；感谢 Tri Cao 提供的演示镜头。同时，我们也感谢 NVIDIA GEAR 团队和 CMU LeCAR 实验室的持续支持。

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. arXiv preprint arXiv:2204.01691, 2022.

[2] Michael Ahn, Debidatta Dwibedi, Chelsea Finn, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Karol Hausman, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, et al. Autort: Embodied foundation models for large scale orchestration of robotic agents. arXiv preprint arXiv:2401.12963, 2024.

[3] Anthropic. Introducing claude opus 4.7. https://www.anthropic.com/news/claude-opus-4-7, 2026.

[4] Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In International Conference on Machine Learning, pages 1577–1594. PMLR, 2023.

[5] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. arXiv preprint arXiv:2503.14734, 2025.

[6] Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. Nature, 624(7992):570–578, 2023.

[7] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. arXiv preprint arXiv:1606.01540, 2016.

[8] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. arXiv preprint arXiv:2212.06817, 2022.

[9] Benjamin Burger, Phillip M Maffettone, Vladimir V Gusev, Catherine M Aitchison, Yang Bai, Xiaoyan Wang, Xiaobo Li, Ben M Alston, Buyi Li, Rob Clowes, et al. A mobile robotic chemist. Nature, 583 (7815):237–241, 2020.

[10] Rémi Cadène, Quentin Gallouédec, Alexander Soare, and Simon Alibert. gym-pusht: A gymnasium environment for PushT. https://github.com/huggingface/gym-pusht, 2024. Version 0.1.6, adapted from Diffusion Policy.

[11] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. In International Conference on Learning Representations, volume 2025, pages 50466–50494, 2025.

[12] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. The International Journal of Robotics Research, 44(10-11):1684–1704, 2025.

[13] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In Proceedings of the 42nd acm sigplan international conference on programming language design and implementation, pages 835–850, 2021.

[14] Martin A Fischler and Robert C Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. Communications of the ACM, 24(6):381–395, 1981.

[15] Max Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Fei-Fei Li, Guanya Shi, et al. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. arXiv preprint arXiv:2603.22435, 2026.

[16] Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J Szostkiewicz, Dmytro Shved, Gavin J Gyimesi, Jon M Laurent, Samantha M Wright, Muhammed T Razzak, et al. A multi-agent system for automating scientific discovery. Nature, pages 1–3, 2026.

[17] Stefan Gottschalk, Ming C Lin, and Dinesh Manocha. Obbtree: A hierarchical structure for rapid interference detection. In Proceedings of the 23rd annual conference on Computer graphics and interactive techniques, pages 171–180, 1996.

[18] Tuomas Haarnoja, Sehoon Ha, Aurick Zhou, Jie Tan, George Tucker, and Sergey Levine. Learning to walk via deep reinforcement learning. arXiv preprint arXiv:1812.11103, 2018.

[19] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, et al. pi*0.6: a vla that learns from experience. arXiv preprint arXiv:2511.14759, 2025.

[20] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. pi05: a vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054, 2025.

[21] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In International Conference on Learning Representations, volume 2024, pages 54107–54157, 2024.

[22] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. https://github.com/karpathy/autoresearch, 2025. GitHub repository.

[23] Ross D King, Kenneth E Whelan, Ffion M Jones, Philip GK Reiser, Christopher H Bryant, Stephen H Muggleton, Douglas B Kell, and Stephen G Oliver. Functional genomic hypothesis generation and experimentation by a robot scientist. Nature, 427(6971):247–252, 2004.

[24] Kun Lei, Huanyu Li, Dongjie Yu, Zhenyu Wei, Lingxiao Guo, Zhennan Jiang, Ziyu Wang, Shiyu Liang, and Huazhe Xu. Rl-100: Performant robotic manipulation with real-world reinforcement learning. arXiv preprint arXiv:2510.14830, 2025.

[25] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In 2023 IEEE International conference on robotics and automation (ICRA), pages 9493–9500. IEEE, 2023.

[26] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292, 2024.

[27] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 16961–16969. IEEE, 2024.

[28] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. Science Robotics, 10(105):eads5033, 2025.

[29] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Augmenting large language models with chemistry tools. Nature machine intelligence, 6(5):525–535, 2024.

[30] Jason Ma, William Liang, Hung-Ju Wang, Yuke Zhu, Linxi Fan, Osbert Bastani, and Dinesh Jayaraman. Dreureka: Language model guided sim-to-real transfer. RSS, 2024.

[31] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Jim Fan, et al. Eureka: Human-level reward design via coding large language models. In International conference on learning Representations, volume 2024, pages 26516–26560, 2024.

[32] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. Advances in neural information processing systems, 36:46534–46594, 2023.

[33] Moonshot AI. Kimi code. https://www.kimi.com/code/en, 2026.

[34] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots. arXiv preprint arXiv:2603.04356, 2026.

[35] OpenAI. Openai codex. https://developers.openai.com/codex/, 2026.

[36] Dean A Pomerleau. Alvinn: An autonomous land vehicle in a neural network. Advances in neural information processing systems, 1, 1988.

[37] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. Advances in neural information processing systems, 36:68539–68551, 2023.

[38] Samuel Schmidgall and Michael Moor. Agentrxiv: Towards collaborative autonomous research. arXiv preprint arXiv:2503.18102, 2025.

[39] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants. Findings of the Association for Computational Linguistics: EMNLP 2025, pages 5977–6043, 2025.

[40] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. Advances in neural information processing systems, 36:8634–8652, 2023.

[41] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. arXiv preprint arXiv:2209.11302, 2022.

[42] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. In Proceedings of the IEEE/CVF international conference on computer vision, pages 11888–11898, 2023.

[43] Simon Thorpe, Denis Fize, and Catherine Marlot. Speed of processing in the human visual system. Nature, 381(6582):520–522, 1996. doi: 10.1038/381520a0.

[44] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023.

[45] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In Forty-first International Conference on Machine Learning, 2024.

[46] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation. arXiv preprint arXiv:2311.01455, 2023.

[47] Jiayi Weng. Learning beyond gradients. https://trinkle23897.github.io/learning-beyond-gradients/, May 2026. Blog post.

[48] Wenli Xiao, Haotian Lin, Andy Peng, Haoru Xue, Tairan He, Yuqi Xie, Fengyuan Hu, Jimmy Wu, Zhengyi Luo, Linxi Fan, et al. Self-improving vision-language-action models with data generation via residual rl. arXiv preprint arXiv:2511.00091, 2025.

[49] Tianbao Xie, Siheng Zhao, Chen Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2reward: Reward shaping with language models for reinforcement learning. In International Conference on Learning Representations, volume 2024, pages 35663–35699, 2024.

[50] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. arXiv preprint arXiv:2504.08066, 2025.

[51] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2022.

[52] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. Advances in neural information processing systems, 36:11809–11822, 2023.

[53] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, et al. Language to rewards for robotic skill synthesis. arXiv preprint arXiv:2306.08647, 2023.

附录

所有视频均可在 research.nvidia.com/labs/gear/enpire获取

附录大纲

A 环境创建 14
A.1 自主复位 . . . 14
A.2 自研驱动的奖励设计 . . . 15
B 机器人系统设置 16
B.1 车队架构 . . . 16
B.2 站点硬件 . . . 17
B.3 低级控制 . . . 18
B.4 每任务配置 . . . 18
B.5 真实世界强化学习系统集成 . . . 18
B.6 针插入创意树 . . . 19
C 物理自研究：消融实验 20
C.1 实验设置 . . . 20
C.2 Token 使用量 . . . 20
C.3 原生视觉能力 . . . 20
C.4 模型与夹具比较 . . . 21
D 仿真基准结果 22
D.1 RoboCasa 仿真接口与 API 表面 . . . 22
D.2 RoboCasa 评估协议 . . . 23

A. 环境创建

A.1. 自主复位

我们证明，给定任务规范，编程智能体可以通过组合感知和规划 API 来合成长时程重置函数，包括用于开放词表目标检测的 SAM3（如Code Snippet 1所示）、用于连续 6-自由度位姿跟踪的 BundleSDF，以及用于 GPU 加速的无碰撞轨迹最优化的 cuRobo。在执行过程中，这些重置函数会消耗实时的 RGBD 流和本体感知状态，共同支持鲁棒的目标定位、抓取姿态生成和运动规划。编程智能体还可命令并读取夹爪力矩信号，作为触觉反馈的替代；我们发现这些信号对于滑移检测以及在高精度要求的操作任务中控制抓握力度至关重要。

Code Snippet 1: **工具 API：SAM3 多对象分割**。从选定的摄像头视角中检测查询对象的所有实例，返回每个实例的掩码、置信度得分以及反投影的 3D 中心点。

```
class SegmentAllObjectsTool(Tool):
    """Detect all instances of an object using SAM3 multi-detection."""
    parameters = [
        ToolParameter("query",              "str"),
        ToolParameter("camera",             "str",   default="top"),
        ToolParameter("score_thresh",       "float", default=0.1),
```

```
        ToolParameter("image_bbox",          "list",  default=None),
        ToolParameter("image_exclude_bboxes", "list",  default=None),
        ToolParameter("filters",              "list",  default=None),
        ToolParameter("rank_by",              "dict",  default=None),
    ]
    # query:
    #     Natural-language description of the object to segment.
    # camera:
    #     Camera view to capture from. One of {top, left, right}.
    # score_thresh:
    #     Minimum SAM3 confidence score. Detections below threshold are dropped.
    # image_bbox:
    #     [x_min, y_min, x_max, y_max] pixel crop. Zeros all pixels outside the
    #     box before sending to SAM3 to suppress background noise. Returned masks
    #     remain in full-image coordinates so centroid_world_xyz back-projects correctly.
    # image_exclude_bboxes:
    #     List of [x_min, y_min, x_max, y_max] boxes to zero out before sending to
    #     SAM3. Inverse of image_bbox — used to blank known distractors (e.g. gripper
    #     jaws). Combinable with image_bbox: image_bbox is applied first, then
    #     exclusions zero out within the kept region.
    # filters:
    #     List of {metric, op, value} dicts AND-ed together to prune detections.
    #     metric in {confidence, area, AR, C, P}; op in {>=, <=}.
    #     AR: aspect ratio; C: compactness 4*pi*area/P^2; P: perimeter in pixels.
    #     Detections with None or non-finite metric values are dropped.
    # rank_by:
    #     {metric, direction} dict controlling sort order of returned detections.
    #     metric in {confidence, area, AR, C, P}; direction in {max, min}.
    #     Default: {confidence, max}.

    def execute(self, **kwargs) -> ToolResult:
        rgb        = self._fetch_camera_image(kwargs["camera"])
        rgb_for_sam = self._preprocess(rgb, kwargs["image_bbox"],
                                       kwargs["image_exclude_bboxes"])
        detections  = requests.post(f"{self._sam3_url}/segment_all",
                      json={"text":            kwargs["query"],
                            "image_b64":       to_b64(rgb_for_sam),
                            "score_threshold":kwargs["score_thresh"]}).json()
        depth, intr, T_cam_world = _fetch_camera_params(self._cam, kwargs["camera"])
        results = [
            SegmentationResult(
                mask              = np.load(io.BytesIO(base64.b64decode(d["mask_b64"]))),
                score             = d["score"],
                bbox_xywh          = d["bbox_xywh"],
                mask_area          = d["mask_area"],
                perimeter_px       = _shape_stats(mask)["perimeter_px"],
                compactness        = _shape_stats(mask)["compactness"],
                centroid_uv        = _shape_stats(mask)["centroid_uv"],
                centroid_world_xyz = _project_centroid(mask, depth, intr, T_cam_world),
                depth_m            = depth[v, u],
            ) for d in detections
        ]
        return ToolResult(success=True,
                          data=_rank_results(_apply_filters(results,
                                             kwargs["filters"]), kwargs["rank_by"]))
```

通过这些 API 工具，编码智能体可以合成感知技能，如语义分割和 3D 边界框生成，以及操作技能，包括拾取与放置、物体重定向、定位和双臂交接。将这些技能与故障检测及重试机制相结合，可生成对现实世界扰动具有鲁棒性的长时程重置函数。Code Snippet 2 展示了 GPU 插入任务的一个具体实例，其中智能体将基于 SAM3 的物体定位、基于 RANSAC (14) 和 OBB (17) 的 3D 边界框估计、力矩验证的抓取以及 cuRobo 无碰撞交接组合成一个完整的重置流水线。中间的分割和边界框输出可视化结果见 Fig. 8。

Code Snippet 2: **重置策略：** GPU **插入恢复**。使用 SAM3 定位主板和目标插槽，采用扭矩验证的抓取方式抓取并取出 GPU，然后通过 cuRobo 无碰撞交接路径将其运送至停放位。

```
def reset_gpu_insertion():
    go_home()

    # --- SAM3: localize motherboard and target PCIe slot ---
    scene = sam3_localize_motherboard(camera="top")
    hover_pos, scene = acquire_slot_hover(scene)

    # --- grasp GPU and extract from slot ---
    descend_guided(scene)
    close_gripper(torque_limit=LEFT_CLOSE_TORQUE_LIMIT)
    verify_grasp_or_raise()                          # gripper torque check
    lift_and_pull(scene)

    # --- handover: carry GPU to parking pose on table ---
    freespace_move(left_target_pos=TABLE_APPROACH_POS,
                   left_target_rpy=TABLE_PLACE_RPY,
                   backend="curobo")
    release_glide(TABLE_PLACE_POS, TABLE_PLACE_RPY)     # synchronized open
    cartesian_retract_after_release()
    go_home()
```

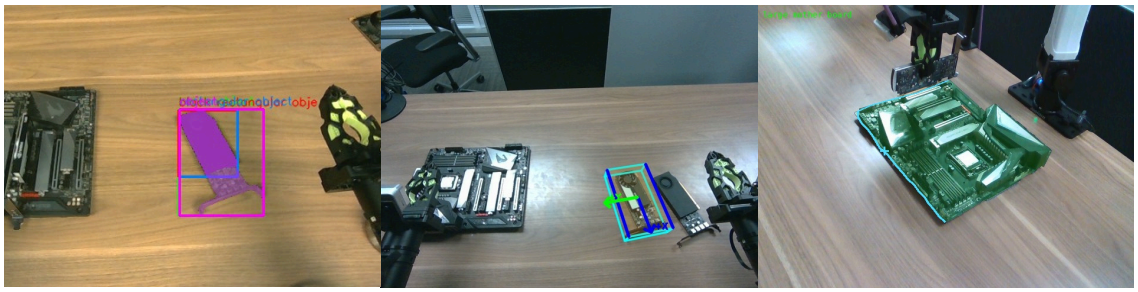


Figure 8: 用于 GPU 插入的可视化工具。由智能体编写的辅助检测工具，通过 SAM3 和深度估计实现 GPU 定位。

A.2. 自研驱动的奖励设计

自动研究可以用于自动化奖励函数的设计。该过程仅需人工参与两个步骤：首先，收集几分钟具有代表性的成功和失败演示；其次，指定奖励设计要求，包括目标准确率、召回率和推理延迟。

例如，我们指派智能体构建一个视觉奖励函数，用于检测扎带是否被系紧。为了评估，我们记录几分钟的标注成功与失败的扎带系紧快照，作为隔离的保留测试集——智能体可以研究失败案例，但不能在测试集上进行训练，也不能修改指标计算方式。智能体自主通过提示搜索探查 SAM3 API，调整裁剪和深度阈值以抑制背景噪声，并通过转换为 ONNX 格式、编译计算图以实现多对象并行前向传播，满足 150 毫秒的延迟预算。决策逻辑最终收敛于双视角几何测试，如Fig. 4所示。简化实现见Code Snippet 3。

自研并非仅限于视觉奖励；编码智能体可以构建融合多种感知模态的混合奖励函数，包括本体感觉和末端执行器受力估计。例如，Fig. 9 展示了一个自主设计的插销插入奖励函数，该函数结合了视觉对齐、插入深度以及接触力信号。

Code Snippet 3: **双视角扎带验证**。通过几何测试结合顶部相机视图和右侧相机视图，判断扎带是否穿过头部，防止单视角假阳性。

```python
def get_reward_from_top_right_cam(rgb_t, rgb_r):
    dets = segment_zip_tie_views(rgb_t, rgb_r, threshold=sam3_score_threshold)
    strap = union([d["mask"] for d in top2_by_score(dets["top"]["strap"])])
    head = top1_by_score(dets["top"]["head"])["mask"]
    strap = binary_dilation(strap, iterations=top_dilate)
    head = binary_dilation(head, iterations=top_dilate)
    overlap = (strap & head).sum() / head.sum()
    protrude = strap[:, :head_left_x(head)].sum() / head.sum()
    top_ok = overlap >= top_inter_frac and protrude >= top_left_mult

    right = binary_dilation(dets["right"]["strap"]["mask"], iterations=right_dilate)
    head_box = bbox_shrunk(dets["right"]["head"], frac=right_shrunk_frac)
    labeled, n_comp = label(right)
    if n_comp == 2:
        right = right | connect_components(labeled)
    right_ok = n_comp in (1, 2) and (right & head_box).any()
    return top_ok and right_ok
```

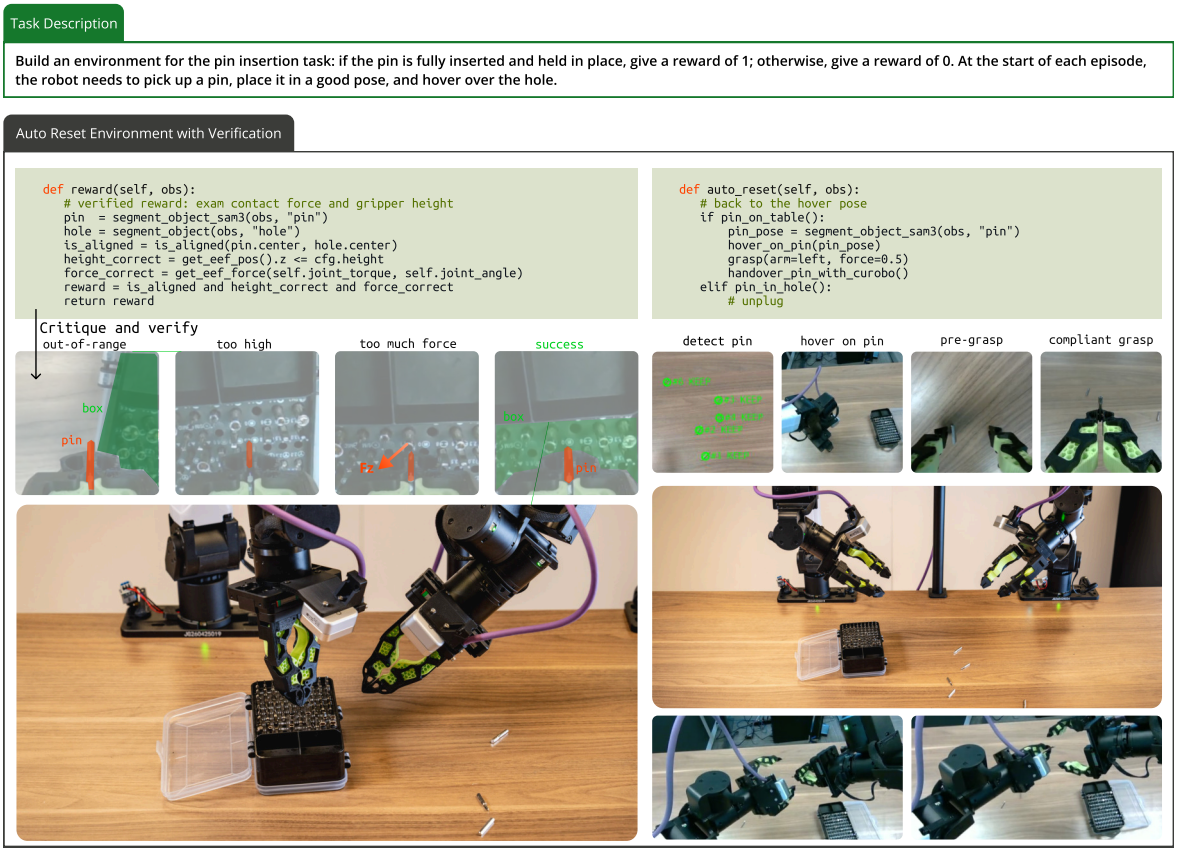


Figure 9: 插针任务的奖励函数。智能体提出一种混合验证策略，融合了视觉对齐（将针尖对准孔洞）、本体感知信号获取的插入深度以及末端执行器扭矩检测。

B. 机器人系统设置

B.1. 车队架构

我们的物理基础设施由**八个双臂机器人工作站**组成，以去中心化的方式运行。每个工作站拥有自己的机械臂和摄像头、计算资源以及专属的编码智能体。各工作站智能体发出的硬件控制请求，均通过本地的 FastAPI 服务器进行路由。各工作站之间的协调完全借助 Git 实现：各站并不向中央服务器流式传输状态，而是通过向共享代码仓库推送（push）和拉取（pull）代码、配置、工具及结果来实现协作，从而使某一站点发现的改进经由常规的版本控制机制传播至其余所有站点。由于全部协调均经由 Git 进行，智能体之间可相互借鉴思路：它们可自由合并其他分支的变更，或挑选性地摘取单个提交（cherry-pick），从而有选择地采纳其他站点所发现的有前景的方法与成果。这使得整个机器人集群保持松耦合且具备容错能力——各工作站可独立运行、发生故障并恢复，而共享的 Git 历史则作为单一可信来源，记录各站点已尝试与学成的内容。

控制端点。
FastAPI 服务器通过一组小型端点暴露给智能体，用于驱动数据采集和执行：/start 在真实硬件上启动一次轨迹收集，/restart 分配一个全新的轨迹缓冲目录，而 /home 则将机械臂返回至原点配置。/restart 端点确保连续的轨迹彼此分离且可单独访问，从而保证不同假设或实验的数据从不混杂：每次轨迹均写入独立目录，智能体能够将其结果归因于所测试的具体变更。这些端点在我们所有的任务中共享使用。Push-T 任务额外提供了 /avoid 和 /resume 端点以处理遮挡问题：/avoid 将机械臂后退出顶部摄像头视野，使其不再遮挡场景，而 /resume 则将机械臂恢复至原始位置以继续操作。

智能体沙箱与上下文。
每个站点的编码智能体在其专属的沙盒环境中运行，该沙盒仅限于单个自研研究仓库。在此沙盒内，智能体享有提升的自主权：动作级权限提示被绕过，使其无需每一步都获得人工批准即可执行命令，并且被授予无限制的互联网访问权限。智能体可获取当前自研研究会话期间收集的完整机器人数据，并可自由利用所有可用信息以达成其目标。对于普通的全新会话，初始化前会清除原始仓库状态、回放数据、检查点以及先前会

/goal Achieve a 100
episodes. Read @auto_research.md and fan out a subagent team to study
different algorithms (RL, BC, hybrid). You may reuse checkpoints or data
buffers across hypotheses. Create your own branch from autorl following the
instructions. Do not directly work on or push to autorl. You must actively
monitor other branches and commits from origin that branched off autorl to
leverage knowledge from those branches. You must actively push your branch to
contribute to the repo and pull from origin. Collaborate with other agents
via Git.

Figure 10: 提供给每个站点编码智能体的自动研究提示。

话的临时日志，确保智能体从一个干净、隔离的工作空间开始。对于迁移实验（如在引脚插入后初始化 GPU 插入），智能体则会收到一个从先前引脚插入自研研究会话中提炼出的显式 Markdown 摘要；尽管如此，原始轨迹、隐藏日志和检查点仍会被移除，因此迁移是通过书面摘要实现的，而非通过对先前会话状态的无限访问。分配给每个站点智能体的提示如Fig. 10所示。

B.2. 站点硬件

所有 8 个站点的硬件完全相同。每个站点包含两个来自 I2RT 的 YAM（Yet Another Manipulator）机械臂，以固定的双臂配置安装，一组摄像头，以及一个运行 FastAPI 服务器、策略推理和站点智能体的工作站。

操纵装置与驱动
每只机械臂均为 6 自由度（DoF）操作臂，配备 1 自由度的平行颚夹持器，每只机械臂有七个驱动关节，双臂共十四个。所有关节均通过 CAN 总线由无刷执行器驱动。六自由度机械臂关节采用带有重力补偿的 PD 控制，而夹持器则运行在力限制模式下，直接限定夹持力（Sec. B.3）；我们利用该夹持器级别的力限制功能，实现稳健抓取以及在整个机群中安全、无需人工值守的运行。

计算。
每个站点均在单一工作站上运行，其规格如 Tab. 1 所列。所有策略推理和站内计算均在此单个 GPU 上完成，不使用共享簇或站外计算。

Table 1: 每个站点的计算规格。

Component	Specification
GPU	1× NVIDIA RTX 5090, 32 GB
CPU	Intel Core Ultra 9 285K, 24 cores
RAM	128 GB
OS	Ubuntu 22.04 LTS
GPU stack	NVIDIA driver 595.58.03, CUDA 13.2

感知。
除非另有说明，感知系统使用英特尔 RealSense D405 相机：一个自顶向下相机用于观测工作区，以及两个安装在机械臂腕部的相机（每条机械臂各一个），用于提供抓取物体和接触区域的近距离视图。一项任务还使用一个侧向安装的 RealSense D435i 以获得更广角的第三人称视角（Sec. B.4）。策略以 30 Hz 运行，其动作目标由下文所述的低层关节控制器跟踪，该控制器通过 CAN 总线以 100 Hz 的频率运行。我们还使用 Viser 实现机器人状态、相机帧和目标位姿的实时浏览器端 3D 可视化，支持对自主运行过程的监控、校准调试以及回合检查。

B.3. 低级控制

策略动作在 30 Hz 下推断，每个动作目标由运行在 100 Hz 的低层关节控制器通过 CAN 总线进行跟踪。两个关节组采用不同的控制方式。六个手臂关节使用**带重力补偿的** PD **控制**，在跟踪命令关节目标的同时，前馈重力项分担了静态负载，使得 PD 增益只需处理残余误差。

1 自由度夹爪以**力矩限值的柔顺抓取**方式运行：夹爪不闭合至一个刚性目标宽度，而是根据设定的力矩限值施加一个有限的抓握力。这种力限制是实现鲁棒且安全抓取的关键。由于手指在物体周围以有限的力自动调整，而非强制移动到固定位置，因此该抓取方式能够适应物体姿态和大小的变化，并对感知和放置误差具有鲁棒性。相同的力限值也限制了系统在抓取、接触和插入过程中所能施加的最大力，从而防止因对准错误或失败而造成夹爪、被操作部件及夹具的损坏。由于整个车队需在全部 8 个工位上自主且无人值守地运行，这种有限力行为至关重要：一旦出现不良接触，系统将安全停机而非造成硬件损伤，且无需人工介入即可完成处理。

B.4. 每任务配置



(a) Four-camera setup (GPU insertion).



(b) Three-camera setup (pin insertion, Push-T, zip tie).

Figure 11: 每站相机配置。所有任务均使用一个自顶向下相机和两个腕部安装的相机，均为 RealSense D405 型号。在Fig. 11a中的 GPU 插入任务额外使用左侧支柱上安装的一个侧向 RealSense D435i 相机，以获得更宽广的第三人称视角来观察插槽；其余任务则使用Fig. 11b中的三相机配置。

四个任务共享站台硬件以及其上的 30 Hz 策略环，仅在相机和控制配置的细微细节上有所不同。如图Fig. 11所示，所有四个任务均使用一个自顶向下相机和两个腕部相机，均为 RealSense D405；其中 GPU 插入任务额外使用一个侧装的 RealSense D435i，以获得更宽广的第三人称视角来观察插槽。

B.5. 真实世界强化学习系统集成

我们集成了 PLD-RL 流水线 (48)，为编码智能体提供一个受控沙盒，以实现基于在线数据采集的算法自动研究。该基础设施遵循 SERL (27; 28) 的异步设计进行开发。我们将其实现为三层分布式系统，将机器人交互、策略学习和推理在独立进程中分离。部署层运行在靠近机器人的控制机上，负责硬件编排、回合记录以及人机协同遥操作。第二层学习器运行在 GPU 主机上，利用预训练视觉骨干网络对像素观测进行编码，训练强化学习智能体（演员 & 评论员）。第三层演员暴露一个 Portal/ZMQ msgpack 兼容端点，使控制器能够通过与脚本化和遥操作控制模式共享的统一协议请求动作。

部署层与学习器之间的数据流通过一个有意设计得较为松散的基于磁盘的协议进行协调。部署层将每个回合按步骤序列化为观测张量和同步的相机视频流（.mp4），存入回放缓冲区目录中，并附带每一步的动作标签，用于标识动作来源（如强化学习、人类等）。同时，一个守护线程（DiskBufferIngestor）会周期性地轮询该目录，解析已最终完成的回合，并根据动作来源标签，按照 RLPD 风格的数据混合协议 (4)：强化学习生成的转移被放入在线经验池，而人类和手动操作的转移则存入一个分离的示范缓冲区，并在每次训练批量中进行混合。这种设计能够灵活地融合来自不同来源的数据。

B.6. 针插入创意树

Fig. 12 将在夹持插入任务上进行的智能体团队自研过程以思想树（上方）和其爬山曲线（下方）的形式可视化。每个结点 I_k 代表团队探索的一个想法；相关联的想法通过水平线连接，每条新分支对应一组新的想法。实心绿色结点标记了提升团队平均最优成功率的想法，而空心结点则表示被评估但未带来改进的想法。粗黑线追踪了得分最高的想法的传承脉络。带圆圈的结点为突出显示的想法，同时也在下方的爬山曲线上进行了

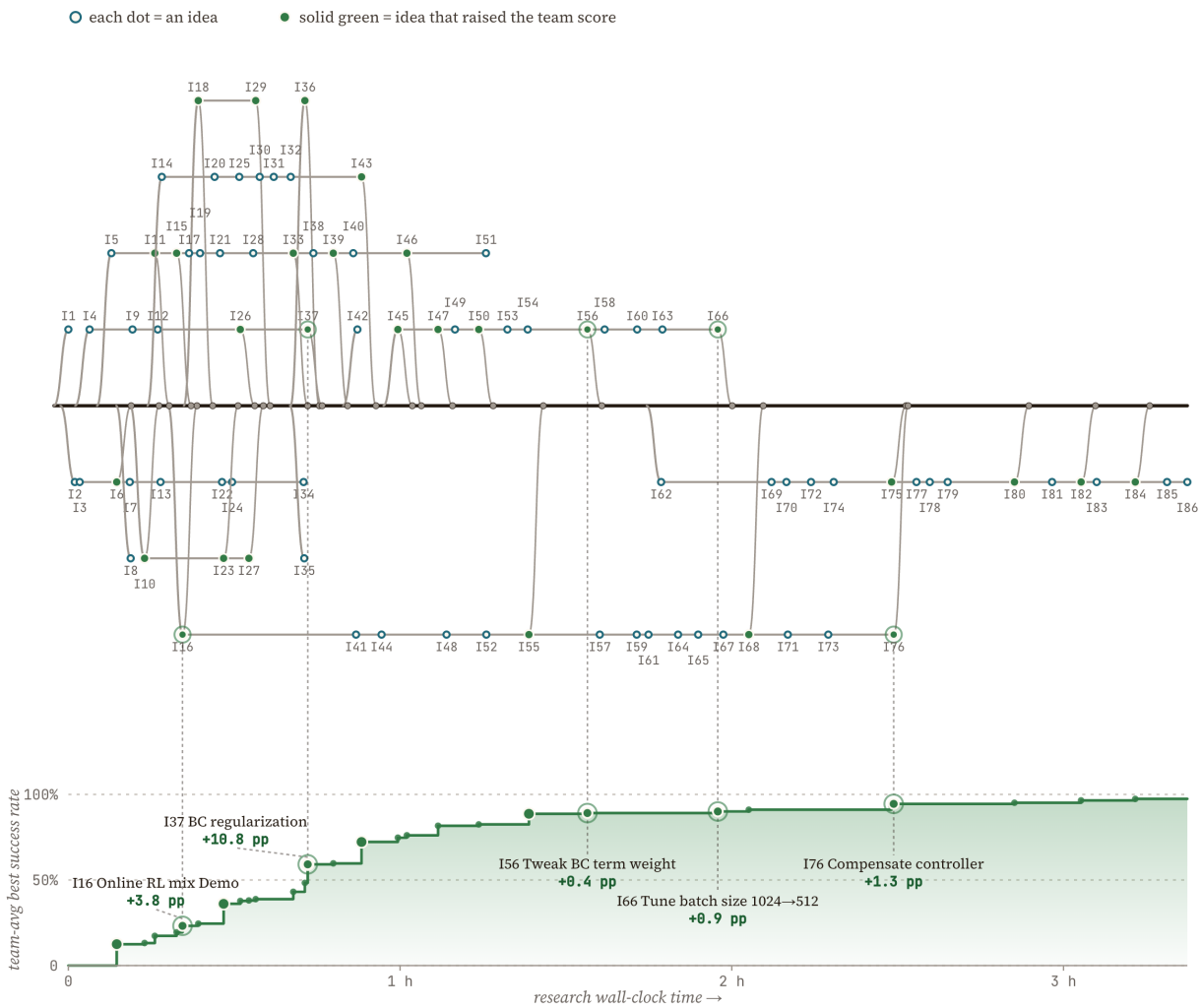


Figure 12: **插针任务的智能体思想树与爬山优化过程**。上图：智能体的思想树，其中每个结点 Ik 是一个提出的思路，每条新分支代表一个新的思路方向。用水平线连接的两个结点表示相关思路。实心绿色结点提升了团队平均最优成功率；空心结点虽被评估但未带来提升。粗黑线追踪了得分最高的思路的传承脉络，带圆圈的结点为突出展示的思路，其通过虚线与下方的里程碑相连。下图：团队平均最优成功率随研究时钟时间的变化，与思想树共享时间轴。标注的里程碑报告了每个突出思路带来的提升百分点（pp）。

标注，并通过虚线与之相连。下方面板报告了团队平均最优成功率随研究时钟时间的变化，与上方思想树共享时间轴。随着智能体通过 Git 协作，少数几个高影响力想法推动了大部分进展——尤其是 BC 正则化（I37, +10.8 pp）——而后期的想法如批量大小调优（I66, +0.9 pp）和控制器补偿（I76, +1.3 pp）则在成功率接近 100% 时贡献了较小的、渐进式的优化。

C. 物理自研究：消融实验

C.1. 实验设置

我们构建了一个简化的现实世界 Push-T 环境 pusht-simple，用于对自研究智能体进行受控消融实验。该任务保留了 Push-T 的核心视觉伺服与接触规划结构，同时减少了回合长度和设置变异性，使其适用于在不同模型、控制框架和视觉定位条件下进行重复对比。与主论文中的 Push-T 任务相比，此变体采用了更宽松的成功边界，使消融实验能够聚焦于控制器发现，而非罕见的终端精度问题。当被推动的 T 块在规定的平移容差范围内到达目标区域，并且其朝向与目标朝向的偏差在 10° 以内时，该回合即视为成功。从操作层面看，智能体必须编写一个控制器，将 T 块旋转约 180° ，并将其放置在扩大的成功区域内部。

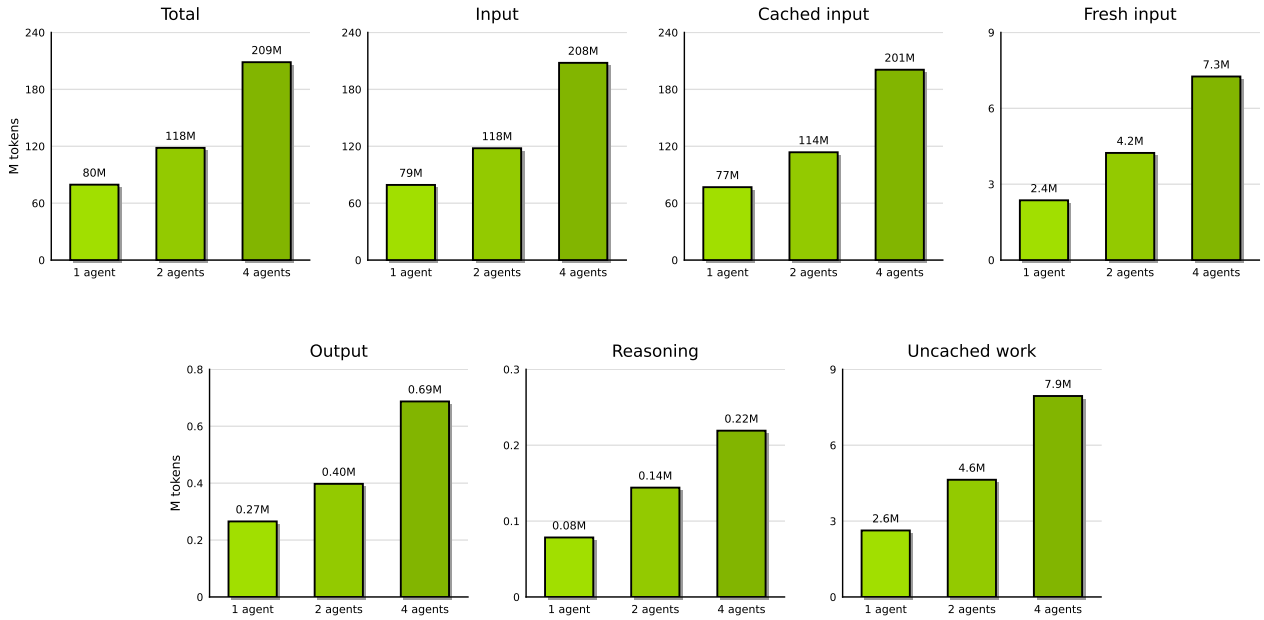


Figure 13: Codex 智能体在简化版真实世界 Push-T 任务中的 token 使用情况分解。每个面板展示了自研究过程中 token 消耗的不同视角，区分了提示/输入、生成/输出以及缓存上下文的使用情况（如可用）。该图通过展示 token 需求在规划、代码编辑、日志检查和重复策略评估迭代中的分布，补充了主论文中汇总的 MTU 分析。

C.2. Token 使用量

在正文中，我们通过汇总输入和输出 token 来总结 token 缩放情况。由于输入、输出和缓存 token 在成本、延迟以及所反映的智能体活动类型方面存在差异，我们进一步按类别报告了 token 使用情况。这种细分有助于明确自动研究预算中有多少用于上下文摄入、代码与计划生成，以及缓存交互历史的重用。为了进行更详细的定性分析，我们可视化了总 token 使用量、输入 token 使用量、缓存输入 token 使用量、新输入 token 使用量、输出 token 使用量以及推理 token 使用量。

C.3. 原生视觉能力

由于 Push-T 任务要求智能体定位物体、推断接触几何关系，并从评估滚动中诊断失败，我们对编码智能体中原生视觉能力的影响进行了消融实验。Codex 工具包支持原生视觉理解，这意味着编码智能体可以直接查看图像。然而，许多开源智能体工具包仅提供文本输入。本实验旨在衡量视觉访问对简化版 Push-T 任务中自动搜索爬山法的影响。除了原始 Codex 配置外，我们还评估了两个基准方案：

1. **无原生视觉的** Codex**；**我们对编码智能体中的图像 token 进行掩码处理，但提供一个可调用的独立图像理解模块。该模块读取图像，生成描述并回答视觉问题；系统提示被修改，以允许编码智能体在需要视觉信息时调用此函数。
2. **无视觉能力的** Codex。我们移除了原生图像流和视觉功能调用。在此情景下，Codex 只能分析基于文本的信息，或编写代码从图像中提取信息。

具备原生视觉能力的编码器首次取得成功。令人惊讶的是，无视觉基准模型在函数调用视觉基准模型之前就取得了成功。这表明，即使没有直接的视觉访问，编码智能体也能从其他日志信号中推断出有用的任务状态，而重复的图像函数调用在任务求解过程中引入了额外开销。

C.4. 模型与夹具比较

我们研究了模型与智能体利用方式的选择对端到端自研性能的影响。我们对比了 Codex 与 GPT-5.5、Claude 与 Opus 4.7，以及 Codex 与 Opus 4.7 的表现。Codex 解决任务速度最快，而使用 Opus API 的 Codex 智能体利用方式在所有测试配置中效率最低。

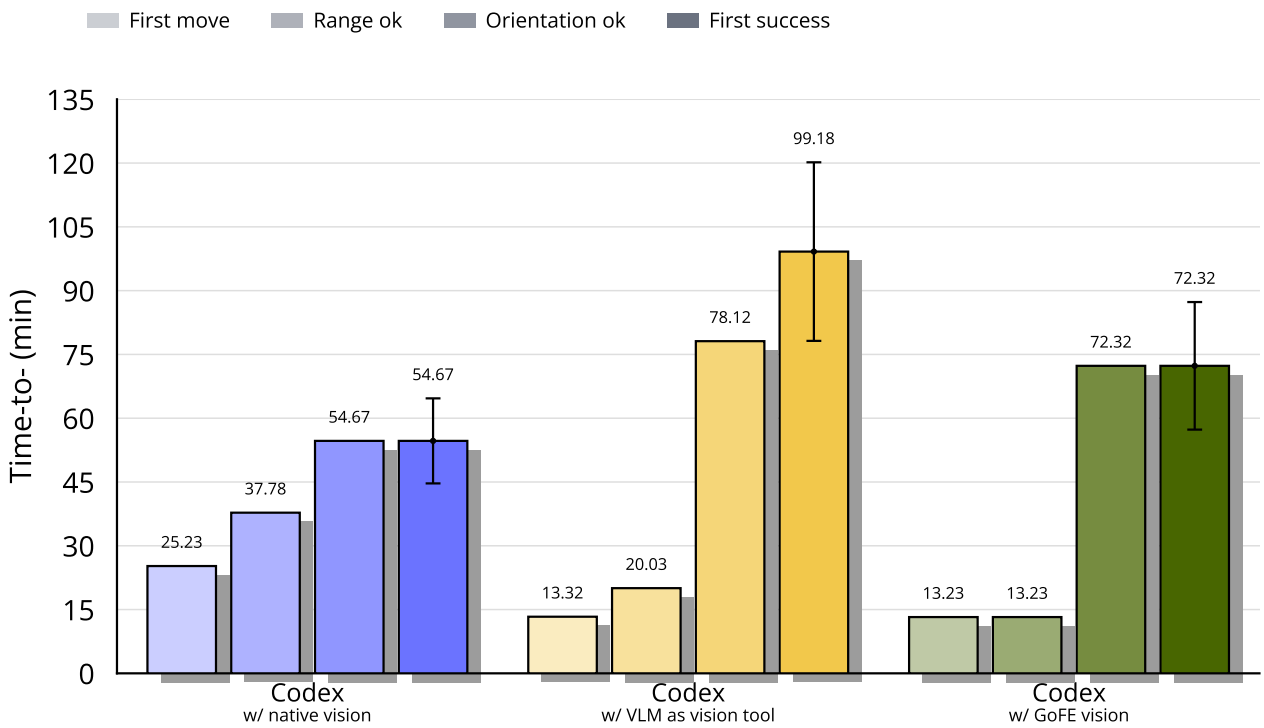


Figure 14: 在不同视觉定位条件下，简化真实世界 Push-T 任务的分阶段进展。

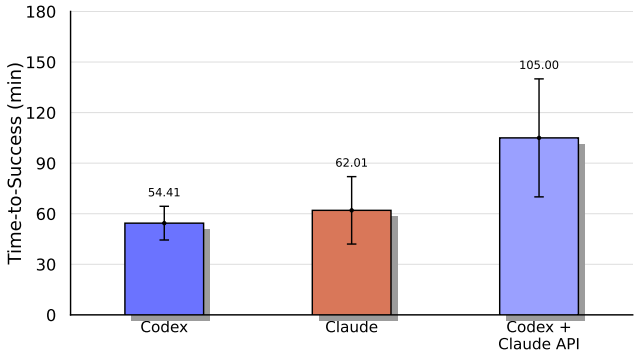


Figure 15: 在简化版真实世界 Push-T 任务中，不同模型与智能体框架配置的时间-成功对比。每种配置通过达到预设成功准则所需的实际运行时间进行评估，该时间既反映了模型的推理能力，也包含了由代码接口、工具访问、执行环及日志检查工作流所带来的实际开销。

D. 仿真基准结果

D.1. RoboCasa 仿真接口与 API 表面

RoboCasa365 基准提供了生成的视觉-语言控制脚本在仿真端的评估情景。我们在 RoboCasa 仿真基础上构建了中层视觉与规划栈，并将模块化和端到端的工具封装为 API。该接口作为单一 Python 命名空间注入生成的视觉脚本中。对于正则评估脚本，会从运行时命名空间中移除泄露特权状态或允许重复重试的 API。特别是，环境禁用了 get_task_info、reset_env、reset_to_initial、任务特定的 CLI 辅助工具以及 oracle 目标查询等 API。oracle 目标 API 仅在正则脚本运行时外部暴露。Tab. 2 总结了接口表面，并标记了每个调用在正则脚本运行时是否可用（✓）或被限制在 CLI/oracle 模式下（C/O）。

Table 2: 我们在评估中使用的 RoboCasa 环境 API 接口。可用性 表示在正则脚本运行时的可用性：✓ 可用；C 仅在非正则命令行界面模式下暴露；O 仅在显式启用 Oracle 接口时暴露。

API	Description	Avail.
State and gripper		
get_robot_state	Joint positions, EEF pose, gripper measurement, and base/arm state	✓
get_gripper_info	Gripper command, measured state, contact bodies, and actuator force	✓
set_gripper	Set normalized gripper command (1 open, 0 closed)	✓
open_gripper	Open the gripper	✓
close_gripper	Close the gripper	✓
Motion		
move_with_curobo	cuRobo plan, OSC waypoint tracking, scene collision	✓
move_with_pyroki	Pyroki IK with SE(3) interpolation, no scene collision	✓
move_with_curobo_joint_exec	cuRobo plan executed in joint-position mode	✓
move_with_pyroki_joint_exec	Pyroki plan executed in joint-position mode	✓
freespace_move	Backward-compatible wrapper over configured backend	✓
move_eef	Move to a direct end-effector target	✓
select_best_grasp	Planner-check grasp candidates and select a feasible one	✓
go_home	Return to the home configuration	✓
nudge	Small local end-effector displacement	✓
nudge_brutal	Small local move with collision avoidance off	✓
rotate_gripper_in_place	In-place wrist rotation	✓
Camera and visualization		
get_camera_image_480_640	RGB image at 480×640	✓
get_camera_depth_480_640	Aligned depth map at 480×640	✓
get_camera_intrinsics_480_640	Intrinsics matched to 480×640	✓
get_camera_image	Legacy RGB image alias at the environment-native resolution	✓
get_camera_depth	Legacy depth alias at the environment-native resolution	✓
get_camera_intrinsics	Legacy intrinsics alias at the environment-native resolution	✓
get_camera_extrinsics	Camera extrinsics	✓
set_debug_markers	Place visualization markers	✓
Perception		
detect_object	Text-conditioned object detection	✓
segment_object	Single-instance segmentation	✓
segment_object_all	Multi-instance segmentation	✓
detect_objects_oneshot	One-shot object detection	✓
sample_grasp_pose_anygrasp	AnyGrasp grasp-pose sampling	✓
vlm_query	Vision-language model query	✓
Planner and collision		
update_planner_world	Set the cuRobo collision geometry	✓
refresh_planner_world	Refresh collision geometry from the scene	✓

API	Description	Avail.
disable_collision_avoid	Disable collision avoidance	✓
Task, policy, and navigation		
get_task_description	Natural-language task instruction	✓
use_policy_output	Query an external policy's output	✓
get_base_state	Mobile-base state	✓
execute_base_trajectory	Execute a planned base trajectory	✓
create_goat_agent	Instantiate a GOAT navigation agent	✓
set_goat_goal	Set the GOAT navigation goal	✓
goat_navigate	Run GOAT navigation	✓
get_goat_state	Return GOAT state and configuration	✓
reset_goat	Reset GOAT state	✓
Utilities		
display_rpy_to_quat	Convert display RPY angles to an xyzw quaternion	✓
np, numpy	NumPy convenience exports for generated scripts	✓
Gated / removed in the canonical script runtime		
get_task_info	Structured task information	C
reset_env	Reset the environment	C
reset_to_initial	Reset to the initial state	C
get_oracle_targets	Ground-truth target pose / oracle query	O

相机与可视化。
相机 API 提供 RGB-D 观测数据、相机内参和外参。我们使用针对分辨率的调用方式，以保持 RGB 图像、深度图和内参之间的相互对齐，同时保留无后缀别名以确保向后兼容性。调试标记仅用于开发过程中可视化中间目标。

运动。
运动 API 支持 cuRobo 和 Pyroki 两种执行方式。cuRobo 使用场景碰撞几何，是自由空间操作的默认选择；而 Pyroki 提供轻量级的 SE(3) 插值与逆运动学，但不包含场景级别的碰撞模型。标准模式通过 OSC 跟踪末端执行器的航点，而在需要精确遵循规划的关节路径时（例如靠近关节极限或狭窄障碍物时），则使用关节执行变体。

感知。
感知 API 提供文本条件检测、分割、one-shot 检测、抓取姿态采样以及视觉语言查询功能。这些调用使生成的脚本能够在不访问模拟器真实值的情况下，将语言指令与视觉观测结果对齐。

规划与碰撞。
规划器 API 会维护模拟场景中的 cuRobo 碰撞几何信息。对于普通的自由空间运动，碰撞避免功能默认开启，而禁用碰撞检测则保留用于短暂的、有意的高接触动作或已知的碰撞模型伪影。

D.2. RoboCasa 评估协议

我们使用上述相同的环境包装器和脚本运行时，在仿真中评估每个 RoboCasa 任务。每个由编码智能体生成的脚本在每个回合中通过正则的评估框架执行一次。该框架创建 RoboCasa 环境，将受限的 Python 工具命名空间注入脚本，运行脚本至完成，记录视频和日志，然后查询环境原生的成功函数以获取最终得分。我们报告二元任务成功情况，该定义基于 RoboCasa 的原生成功判定：当执行结束时，此判定条件返回真，则认为该回合成功。

固定种子和布局。
每次报告的 40 回合评估中，我们使用一个固定的种子三元组列表。每个三元组包含一个 RoboCasa 随机种子、一个布局标识符和一个风格标识符：(seed, layout_id, style_id)。这 40 个三元组通过一个确定性的种子列表生成器一次性生成，生成器种子为 42，之后保存为静态评估文件。对于所有报告的评估（包括较小的诊断子集），我们都使用该保存文件中的行，而不是重新生成新列表，因此子集均为同一正则 40 回合基准的前缀或显式选择。同一三元组列表用于任务中所有对比方法。

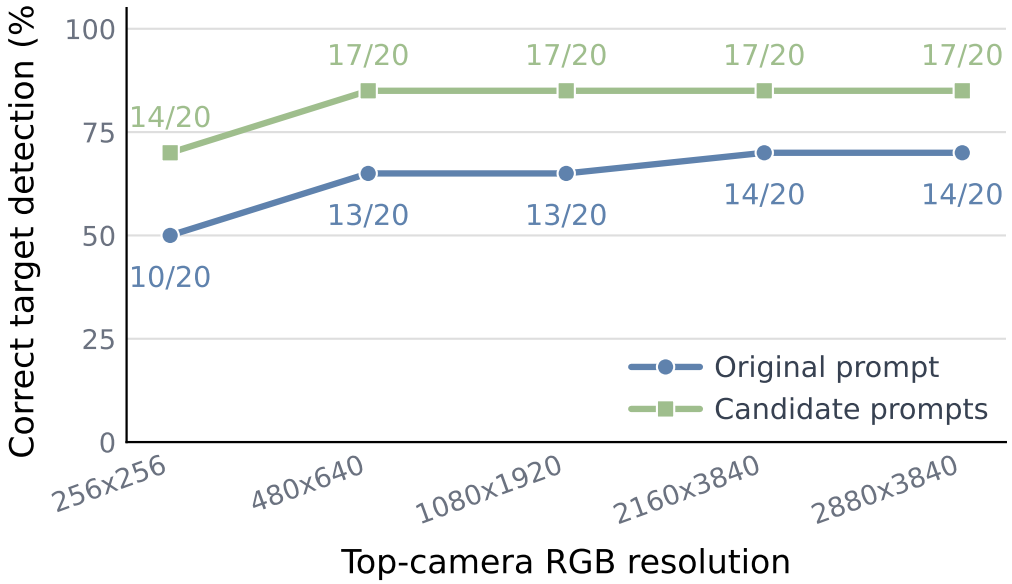


Figure 16: SAM3 在 RoboCasa 桌面到橱柜场景中的目标检测准确率随顶部相机 RGB 分辨率的变化。每个点评估 20 次目标对象查询。原始任务提示在 256×256 时准确率为 10/20，而在更高分辨率下提升至 14/20；而由编程智能体生成的候选提示则从 14/20 提升至 17/20 后趋于稳定。

与 GR00T 的公平比较。
在将生成的视觉脚本与 GR00T 进行比较时，我们使用相同的任务名称、相同的种子三元组、相同的初始模拟器状态、相同的相机配置以及相同的 RoboCasa 成功判定谓词。因此，GR00T 和视觉脚本均在匹配的回合上进行评分，而非在独立采样的场景上。任务语言提示与 RoboCasa 指令完全相同，且两种方法在执行过程中均未获得目标位姿或模拟器物体状态的先验信息。两者的区别在于执行路径：GR00T 通过其学成策略进行滚动执行，而生成的脚本则组合了 Tab. 2 中列出的公开感知、规划和操作 API。我们报告的是在相同固定评估集上的成功率，以便差异能够反映策略或脚本行为本身，而非不同的场景采样或特权信息的影响。

发现策略演示。
除了主论文中报告的匹配种子成功率外，项目网站 research.nvidia.com/labs/gear/enpire还包含了由编码智能体发现的策略的定性演示。特别是，视频展示了智能体发现的策略：结合检测与运动规划，在抓取目标物体之前，先将夹爪移动到目标物体上方的悬停位置，这正是主论文中从 RoboCasa 转移到真实世界剪刀和扎带任务的策略。

结果分析
我们发现了一个感知瓶颈：SAM3 对于 RoboCasa 中较小或模糊的物体，可能返回错误的掩码，甚至无法返回任何可用的掩码。Figs. 16 and 18 通过在相同的“台面至橱柜”诊断数据集上改变图像分辨率和提示词表述来隔离这一效应。将顶视摄像头的分辨率从 256×256 提高至 480×640 可提升检测效果，但仅靠提高分辨率并不能解决所有失败情形。若允许编码智能体重写物体提示词，则可进一步提升目标检测性能，并恢复若干原本因任务原始表述导致生成干扰性掩码的情形。该分析表明，所生成的 RoboCasa 脚本不仅受限于规划与控制能力，还受限于向智能体开放的感知 API 的可靠性；因此，在智能体仿真端的工具使用中，提示词搜索与分辨率选择均为有益环节。

target object	256x256	480x640	1080x1920	2160x3840	2880x3840
cream cheese stick	cheese stick	stick	stick	stick	stick
water bottle	bottle	bottle	bottle	bottle	bottle
saucepan lid	wrong	wrong	lid	lid	lid
ice cube	small cube		small cube		
lemon wedge	wrong	lemon slice	wrong	wrong	wrong
garlic	wrong	small garlic			
colander	metal colander				
sugar cube	wrong	wrong	wrong	wrong	wrong
shrimp			fail	fail	
yogurt	wrong	wrong			
cookie dough ball	wrong				
marshmallow					fail
peeler					
mango					
tomato					
bread					
candle					
cupcake					
lemon					
mushroom					

Original prompt correct　Candidate prompt correct　Wrong mask　No correct mask

Figure 17: 针对 RoboCasa 台面到柜子诊断数据集，在不同顶摄像头分辨率和提示变体下的每对象 SAM3 检测结果。绿色单元表示原始目标提示产生了正确的掩码，黄色单元表示候选提示产生了正确的掩码，灰色单元表示产生了错误的掩码，红色单元表示未返回正确掩码。



Figure 18: 20 个 RoboCasa 桌面到橱柜目标物体在 480 × 640 分辨率下的定性 SAM3 掩码输出。黄色框显示选定的目标掩码；标签指示使用的是原始提示、候选提示还是错误的掩码。